

FinShare — Forgot Password Backend Requirement

Note for Abhilash · 03 April 2026

Hi Abhilash, great news — we have now tested the full auth flow and Registration, Login, and Logout are all working perfectly. Well done! There is just one remaining piece needed to complete the authentication journey: the Forgot Password feature.

Current Auth Status

Feature	Frontend	Backend	Status
Register	✅ Built & tested	✅ POST /api/Auth/register	Fully working
Login	✅ Built & tested	✅ POST /api/Auth/login	Fully working
Logout	✅ Built	No backend needed (clears JWT token locally)	Working — see note below
Forgot Password	✅ Screen built (shows 'coming soon')	❌ Not yet built	Waiting on backend
Reset Password	⌚ Pending	❌ Not yet built	Waiting on backend

Note on Logout:

Logout does not need any backend endpoint. It works by clearing the JWT token from the user's device (localStorage). When the token is gone, the app treats the user as signed out and redirects to /login. The JWT simply expires naturally on its own — no server-side invalidation is needed for this app at this stage.

What Is Needed — Two New Endpoints

The Forgot Password flow has two steps: first the user requests a reset link by entering their email, then they use the link to set a new password. Each step needs one endpoint.

Endpoint 1 — Request a Reset Link

POST /api/Auth/forgot-password

Request body:

```
{ email: string }
```

What it must do:

1. Look up the user by email address in the Users table.
2. If found: generate a secure random token (e.g. Guid.NewGuid().ToString("N"))

or a cryptographic random string).

3. Store the token and an expiry (1 hour from now) against the user record.
4. Send an email to the user with a reset link in this format:
`https://finshare.me/reset-password?token=XXXXXXX`
5. Return success regardless of whether the email exists (security best practice — never confirm whether an email is registered).

Response (always):

```
{ success: true, message: 'If your email is registered, a reset link has been sent.' }
```

Endpoint 2 — Submit a New Password

POST /api/Auth/reset-password

Request body:

```
{ token: string, newPassword: string }
```

What it must do:

1. Look up the token in the Users table.
2. Check the token exists and has not expired (`PasswordResetExpiry > DateTime.UtcNow`).
3. If valid: hash the new password using BCrypt (same as registration) and update the user's password hash in the database.
4. Clear the token and expiry fields so the link cannot be reused.
5. Return success.

Response on success:

```
{ success: true, message: 'Password updated successfully. Please sign in.' }
```

Response on failure (expired or invalid token):

```
{ success: false, message: 'This reset link has expired or is invalid.  
Please request a new one.' }
```

Database Change — Two New Columns on Users Table

You will need to store the reset token and its expiry time. The cleanest way is to add two nullable columns to the existing Users table:

```
ALTER TABLE Users
ADD PasswordResetToken NVARCHAR(200) NULL,
    PasswordResetExpiry DATETIME NULL;
```

Both are nullable. They are NULL when there is no active reset request. They are cleared back to NULL after the password is successfully reset.

Stored Procedures (following existing Dapper + SP pattern)

Add two new stored procedures to keep consistency with the rest of the project:

sp_SavePasswordResetToken

```
CREATE PROCEDURE sp_SavePasswordResetToken
    @Email          NVARCHAR(255),
    @ResetToken     NVARCHAR(200),
    @ExpiryUTC      DATETIME
AS BEGIN
    UPDATE Users
        SET PasswordResetToken = @ResetToken,
            PasswordResetExpiry = @ExpiryUTC
    WHERE UserName = @Email;
END
```

sp_ResetPassword

```
CREATE PROCEDURE sp_ResetPassword
    @Token          NVARCHAR(200),
    @NewPasswordHash NVARCHAR(255)
AS BEGIN
    -- Only update if token exists and has not expired
    UPDATE Users
        SET PasswordHash          = @NewPasswordHash,
            PasswordResetToken    = NULL,
            PasswordResetExpiry   = NULL
    WHERE PasswordResetToken = @Token
        AND PasswordResetExpiry > GETUTCDATE();

    -- Return rows affected: 1 = success, 0 = token invalid/expired
    SELECT @@ROWCOUNT AS RowsUpdated;
END
```

Email Service — Required for Sending the Reset Link

The backend needs an email service to send the reset link. Here are three options — any of them will work:

Option	Setup effort	Cost	Recommendation
--------	--------------	------	----------------

SendGrid	Create free account, add API key to appsettings.json	Free up to 100 emails/day	✅ Best choice — simple, reliable, good .NET SDK available
Gmail SMTP	Use an app password from your Gmail account	Free	⚠️ Works but may be blocked by corporate filters
Azure Communication Services	If already on Azure, can enable from portal	Pay per send (very cheap)	Good if already on Azure

The reset email body should look like this:

Subject: FinShare — Reset your password

Hi [MemberName],

We received a request to reset your FinShare password.

Click the link below to set a new password. This link expires in 1 hour.

👉 <https://finshare.me/reset-password?token=XXXXXXX>

If you did not request this, you can safely ignore this email.

Your password will not change.

— The FinShare Team

Summary — What Manus Will Do Once This Is Ready

Once these two endpoints are live, Manus will update the Forgot Password screen from 'coming soon' to a fully working flow:

- User enters email → frontend calls POST /api/Auth/forgot-password
- Screen shows 'Check your inbox — a reset link has been sent'
- User clicks link in email → browser opens /reset-password?token=XXXX
- New screen shows two password fields (New Password + Confirm)
- On submit → frontend calls POST /api/Auth/reset-password with the token
- On success → navigate to /login with a 'Password updated — please sign in' toast

The frontend Reset Password screen (/reset-password) also needs to be built by Manus — it reads the token from the URL query string and submits it with the new password. Manus will build this as soon as the endpoints are confirmed.

— Joseph